



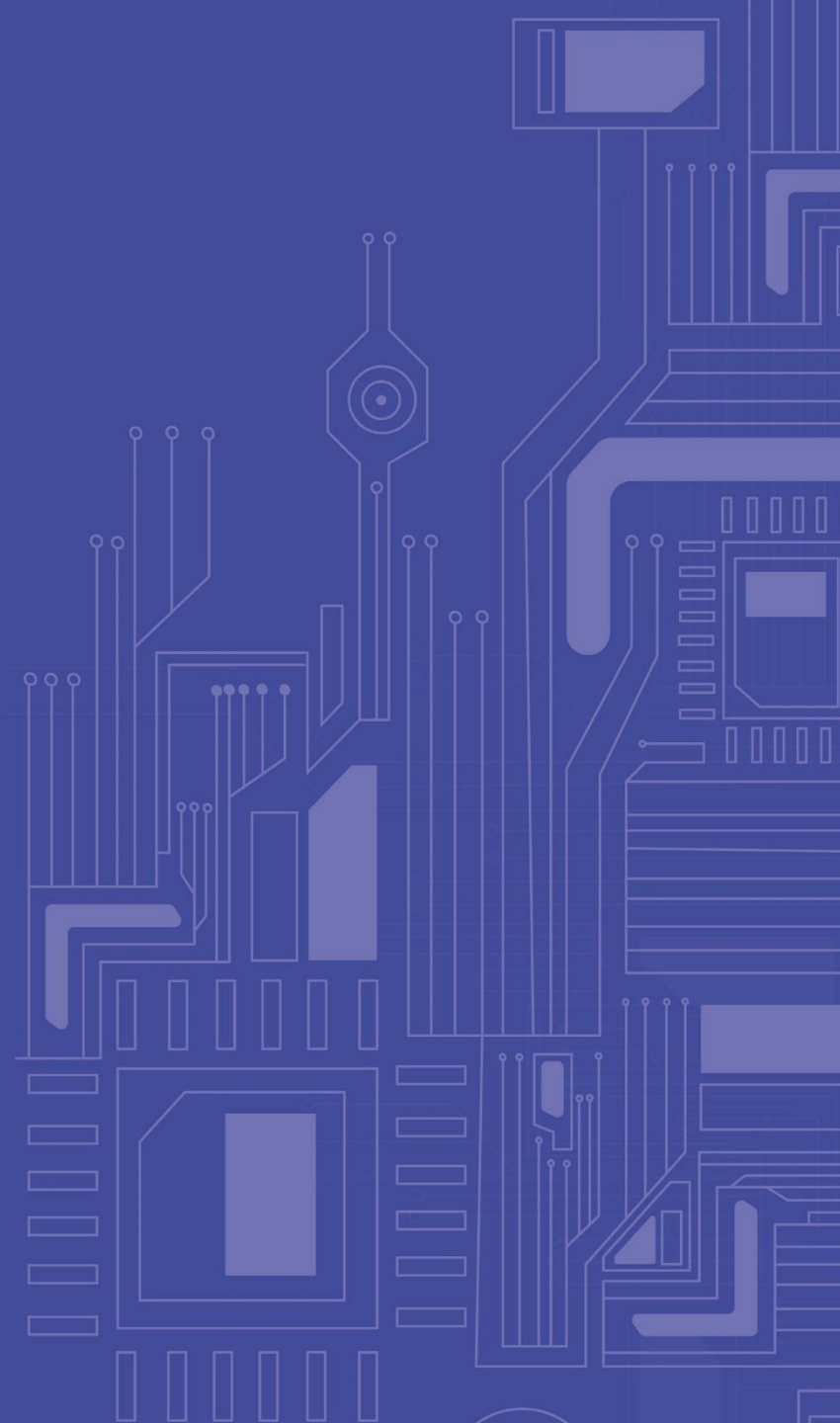
МИНОБРНАУКИ
РОССИИ



Передовые
инженерные
школы

СИСТЕМЫ УПРАВЛЕНИЯ БАЗАМИ ДАННЫХ

Лекция 14



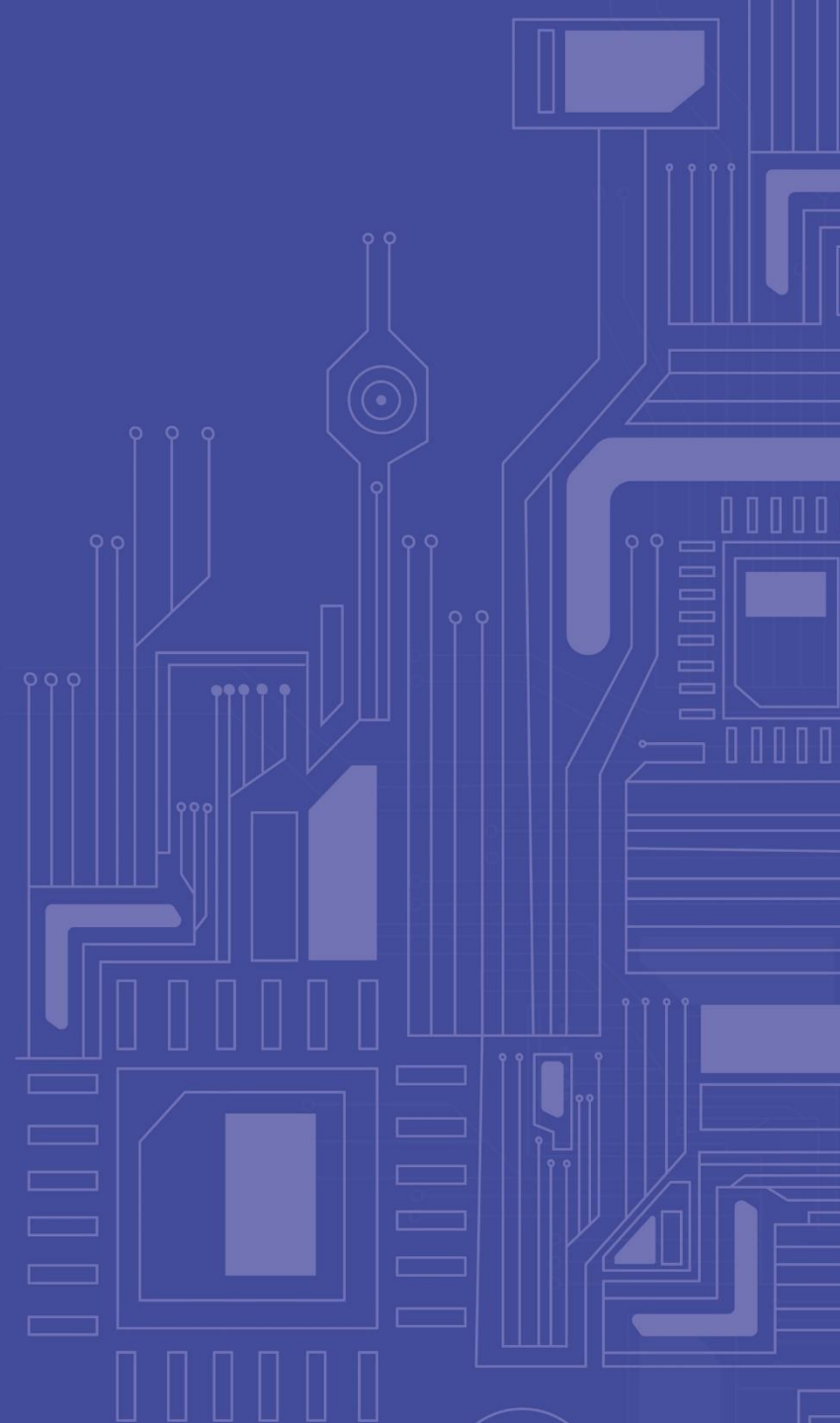
- Операция обновления представлений

- Триггеры

- Поддержка авторизации доступов к данным в SQL

- Управление транзакциями в SQL

ОПЕРАЦИЯ ОБНОВЛЕНИЯ ПРЕДСТАВЛЕНИЙ



ТРЕБОВАНИЯ СТАНДАРТА SQL:1992



Представление – это сохраняемое в каталоге базы данных выражение запросов, обладающее собственным именем и, возможно, собственными именами столбцов.

Синтаксис создания представления:

```
create_view ::= CREATE [ RECURSIVE ] VIEW table_name  
               [ column_name_comma_list ]  
               AS query_expression  
               [ WITH [ CASCADED | LOCAL ] CHECK OPTION ]
```

Операция обновления над представлением должна однозначно отображаться в одну или несколько операций обновления над теми постоянно хранимыми базовыми таблицами, над которыми прямо или косвенно определено данное представление.

В стандарте SQL/92 спецификация запроса считалась допускающей операции обновления в том и только в том случае, когда выполнялись следующие условия:

- в разделе `SELECT` спецификации запроса отсутствует ключевое слово `DISTINCT` (т.е. не требуется удаление строк-дубликатов из результата запроса);
- все элементы списка выборки раздела `SELECT` являются именами столбцов, и ни одно имя столбца не встречается в этом списке более одного раза;
- в разделе `FROM` присутствует только одна ссылка на таблицу, и она указывает либо на базовую таблицу, либо на порождаемую таблицу, допускающую операции обновления;
- прямые или косвенные ссылки на базовую таблицу, прямо или косвенно идентифицируемую ссылкой на таблицу в разделе `FROM`, не встречаются в разделе `FROM` ни одного подзапроса, участвующего в разделе `WHERE` спецификации запроса;
- в спецификации запроса отсутствуют разделы `GROUP BY` и `HAVING`.

ПРИМЕР ДЛЯ СТАНДАТА SQL:1992



```
SELECT EMP_SAL
FROM EMP
WHERE EMP_NAME = (SELECT EMP_NAME
                   FROM EMP
                   WHERE EMP_NO = 4425 )
                   AND DEPT_NO <> 630;
```

Предположим, что с данной спецификацией запроса связано представление с именем **EMPSAL**. Тогда операция

UPDATE EMPSAL SET EMP_SAL = EMP_SAL – 1000.00;

эквивалентна операции

```
UPDATE EMP SET EMP_SAL = EMP_SAL - 1000.00
WHERE EMP_NAME = (SELECT EMP_NAME
                  FROM EMP
                  WHERE EMP_NO = 4425 )
                  AND DEPT_NO <> 630;
```

Операция

DELETE FROM EMPSAL WHERE EMP_SAL > 20000.00;

эквивалентна операции

```
DELETE EMPSAL
WHERE EMP_SAL > 20000.00 AND
      EMP_NAME = (SELECT EMP_NAME
                  FROM EMP
                  WHERE EMP_NO = 4425 )
                  AND DEPT_NO <> 630;
```

В стандарте SQL:1999 правила применимости операций обновления к спецификации запроса существенно уточнены.

Критерии применимости операций обновления

Введены понятия потенциальной применимости операций обновления, применимости операций обновления, простой применимости операций обновления и применимости операции вставки. К спецификации запроса потенциально применимы операции обновления в том и только в том случае, когда выполняются следующие условия:

- в разделе `SELECT` спецификации запроса отсутствует ключевое слово `DISTINCT`;
- элемент списка выборки раздела `SELECT`, состоящий из ссылки на некоторый столбец, не может присутствовать в этом списке более одного раза;
- в спецификации запроса отсутствуют разделы `GROUP BY` и `HAVING`.

Если выражение запросов отвечает условиям потенциальной применимости операций обновления и в его разделе `FROM` присутствует только одна ссылка на таблицу, то к каждому столбцу выражения запроса, соответствующему одному столбцу таблицы из раздела `FROM`, применимы операции обновления. Если выражение запроса отвечает условиям потенциальной применимости операций обновления, но в его разделе `FROM` присутствуют две или более ссылки на таблицы, то операции обновления применимы к столбцу выражения запросов только при выполнении следующих условий:

- столбец порождается из столбца только одной таблицы из раздела `FROM`;
- эта таблица используется в выражении запросов таким образом, что сохраняются свойства ее первичного и всех возможных ключей.

Другими словами, к столбцу таблицы, которая отвечает условиям потенциальной применимости операций обновления, применимы операции обновления только в том случае, когда этот столбец может быть однозначно сопоставлен с единственным столбцом единственной таблицы, участвующей в выражении запроса, и каждая строка выражения запроса может быть однозначно сопоставлена с единственной строкой данной таблицы.

Выражение запросов удовлетворяет условию *применимости операций обновления*, если по крайней мере к одному столбцу выражения запросов применимы операции обновления. Выражение запросов удовлетворяет условию *простой применимости операций обновления*, если в разделе FROM выражения запросов содержится ссылка только на одну таблицу, и все столбцы выражения запросов удовлетворяют условию применимости операций обновления.

Выражение запросов удовлетворит условию *применимости операций вставки*, если оно удовлетворяет условию применимости операций обновления; каждая из таблиц, от которых зависит это выражение (т.е. таблиц, на которые имеются ссылки в разделе FROM), удовлетворяет условию применимости операций вставки и выражение запросов не содержит операций UNION, INTERSECT и EXCEPT. Конечно, это определение базируется на том факте, что для любой базовой таблицы условие применимости операции вставки удовлетворяется.

Приведенный набор правил считается достаточно грубым. В стандарте SQL:1999 он уточняется набором дополнительных правил, устанавливающих восприимчивость различных языковых конструкций к операциям обновления и вставки. В основе этих правил лежит понятие *функциональной зависимости*.

ОБНОВЛЕНИЕ VIEW В POSTGRESQL



In PostgreSQL, a view is a named query stored in the database server. A view can be updatable if it meets certain conditions. This means that you can insert, update, or delete data from the underlying tables via the view.

A view is updatable when it meets the following conditions:

1. The defining query of the view must have exactly one entry in the FROM clause, which can be a table or another updatable view.
2. The defining query must not contain one of the following clauses at the top level:
3. The selection list of the defining query must not contain any:
 - [Window functions](#)
 - [Set-returning function](#)
 - [Aggregate functions](#)

• GROUP BY	• DISTINCT
• HAVING	• UNION
• LIMIT	• INTERSECT
• OFFSET FETCH	• EXCEPT

An updatable view may contain both updatable and non-updatable columns. If you attempt to modify a non-updatable column, PostgreSQL will raise an error.

When you execute a modification statement such as INSERT, UPDATE, or DELETE to an updatable view, PostgreSQL will convert this statement into the corresponding statement of the underlying table.

If you have a WHERE condition in the defining query of a view, you still can update or delete the rows that are not visible through the view. However, if you want to avoid this, you can use the WITH CHECK OPTION to define the view.

РАЗДЕЛ WITH CHECK POINT



ПРИМЕР

EMP

EMP_NO	DEPT_NO	EMP_BDATE	EMP_SAL
2440	1	1950	15000.00
2441	1	1950	16000.00
2442	1	1960	14000.00
2443	1	1960	19000.00
2444	2	1950	17000.00
2445	2	1950	16000.00
2446	2	1960	14000.00
2447	2	1960	20000.00
2448	3	1950	18000.00
2449	3	1950	13000.00
2450	3	1960	21000.00
2451	3	1960	22000.00

Предположим, что в базе данных имеется представление RICH_EMP, определенное следующим образом:

```
CREATE VIEW RICH_EMP AS  
SELECT *  
FROM EMP  
WHERE EMP_SAL > 18000.00;
```

Как видно, в таблице EMP содержится строка, которая соответствует служащему с номером 2447, получающему зарплату в размере 20000 руб. Естественно, эта строка будет присутствовать в виртуальной таблице RICH_EMP. Поэтому можно было бы выполнить, например, операцию

```
UPDATE RICH_EMP  
SET EMP_SAL = EMP_SAL - 3000  
WHERE EMP_NO = 2447;
```

Но если выполнение такой операции действительно допускается, то в результате строка, соответствующая служащему с номером 2447, исчезнет из виртуальной таблицы RICH_EMP!

Чтобы избежать такого противоречивого поведения представляемых таблиц, нужно включать в определение представления раздел **WITH CHECK OPTION**. При наличии этого раздела до реального выполнения операций модификации или вставки строк через представление для каждой строки будет проверяться, что она соответствует условиям представления.

В полном виде синтаксис раздела WITH CHECK OPTION может включать ключевые слова CASCADED или LOCAL.

Предположим, что представление V2 определяется над представлением V1 следующим образом:

Пусть над V2 выполняется некоторая операция O обновления базы данных. Тогда:

```
CREATE VIEW V2 AS  
  SELECT ...  
  FROM V1  
  WHERE ...  
  [ WITH [ CASCADED | LOCAL ] CHECK OPTION ]
```

- если представление V2 определялось без раздела WITH CHECK OPTION, то при выполнении операции O будут проверяться все условия, определяющие ограничения целостности V1 (если в определении V1 присутствовал раздел WITH CHECK OPTION), но никаким образом не будут учитываться условия выборки, содержащееся в выражении запросов представления V2;
- если в определении представления V2 содержался раздел WITH LOCAL CHECK OPTION, то при выполнении операции O будут проверяться все условия, определяющие ограничения целостности V1, и все условия, содержащееся в выражении запросов представления V2;
- наконец, если в определении представления V2 содержался раздел WITH CASCADED CHECK OPTION, то при выполнении операции O будут проверяться все условия, определяющие ограничения целостности V1 (так, как если бы в определении V1 присутствовал раздел WITH CASCADED CHECK OPTION). Тем самым, будут проверяться все ограничения целостности, установленные для всех базовых таблиц, на которых основывается определение V1; все условия всех представлений, определенных над этими базовыми таблицами; и, конечно, все условия, содержащиеся в выражении запросов представления V2.

ПРИМЕРЫ WITH CHECK POINT (1/5)



Чтобы пояснить результаты действия раздела WITH CHECK OPTION, допустим, что в базе данных присутствуют определения двух представлений MIDDLE_RICH_EMP и MORE_RICH_EMP:

```
CREATE VIEW MIDDLE_RICH_EMP AS
  SELECT *
    FROM EMP
   WHERE EMP_SAL < 20000.00
  [ WITH [ CASCADED | LOCAL ] CHECK OPTION ];

CREATE VIEW MORE_RICH_EMP AS
  SELECT *
    FROM MIDDLE_RICH_EMP
   WHERE EMP_SAL > 18000.00
  [ WITH [ CASCADED | LOCAL ] CHECK OPTION ];
```

В каждом из представлений MIDDLE_RICH_EMP и MORE_RICH_EMP может отсутствовать или присутствовать (в одном из двух видов) раздел WITH CHECK OPTION. В совокупности возможен один из девяти случаев:

MORE_RICH_EMP	none	LOCAL	CASCADED
MIDDLE_RICH_EMP			
none	Случай 1	Случай 2	Случай 3
LOCAL	Случай 4	Случай 5	Случай 6
CASCADED	Случай 7	Случай 8	Случай 9

MIDDLE_RICH_EMP			
EMP_NO	DEPT_NO	EMP_BDATE	EMP_SAL
2440	1	1950	15000.00
2441	1	1950	16000.00
2442	1	1960	14000.00
2443	1	1960	19000.00
2444	2	1950	17000.00
2445	2	1950	16000.00
2446	2	1960	14000.00
2448	3	1950	18000.00
2449	3	1950	13000.00

MORE_RICH_EMP			
EMP_NO	DEPT_NO	EMP_BDATE	EMP_SAL
2443	1	1960	19000.00

ПРИМЕРЫ WITH CHECK POINT (2/5)



```
UPDATE MORE_RICH_EMP  
SET EMP_SAL = EMP_SAL + 7000.00;
```

U1

```
UPDATE MORE_RICH_EMP  
SET EMP_SAL = EMP_SAL - 7000.00;
```

U2

Случай 1. Ни в одном из представлений не содержится раздел WITH CHECK OPTION.

Первый неожиданный результат состоит в том, что после выполнения операции U1 тело представления MORE_RICH_EMP оказывается пустым. Действительно, у единственной строки таблицы EMP (со значением EMP_NO, равным 2443), одновременно удовлетворяющей условиям обоих представлений, столбец EMP_SAL принимает значение 26000.00. После этого строка перестает удовлетворять условию представления MIDDLE_RICH_EMP и исчезает из результирующей таблицы MORE_RICH_EMP. Этот результат может быть особенно неожиданным для пользователей базы данных, которым известно, что условие представления MORE_RICH_EMP имеет вид $EMP_SAL > 18000.00$, и соблюдение этого условия должно сохраняться при увеличении размера зарплаты.

Выполнение операции U2 также приведет к опустошению тела MORE_RICH_EMP (в базовой таблице EMP не останется ни одной строки, удовлетворяющей условию этого представления). Возможно, это будет достаточно естественно для пользователей представления MORE_RICH_EMP, которым известно условие представления, но те, кто работает с представлением MIDDLE_RICH_EMP, с удивлением обнаружат в теле результирующей таблицы новые строки.

Случай 2. В определении представления MIDDLE_RICH_EMP содержится раздел WITH LOCAL CHECK OPTION, а в определении MORE_RICH_EMP раздел WITH CHECK OPTION отсутствует.

В этом случае, в соответствии с первыми двумя правилами проверки корректности выполнения операций обновления над представлениями, операция U1 должна быть отвергнута системой (поскольку ее выполнение нарушает условие представления MIDDLE_RICH_EMP). Но заметим, что такое поведение системы будет совершенно неожиданным и непонятным для тех пользователей базы данных, которым известно только определение «верхнего» представления MORE_RICH_EMP, поскольку операция U1 явно не может нарушить видимое ими ограничение.

С другой стороны, операция U2 будет успешно выполнена и по-прежнему приведет к опустошению тела результирующей таблицы представления MORE_RICH_EMP.

Случай 3. В определении представления MIDDLE_RICH_EMP содержится раздел WITH CASCADED CHECK OPTION, а в определении MORE_RICH_EMP раздел WITH CHECK OPTION отсутствует.

В этой ситуации будут проверяться условия, содержащиеся в определении представления MIDDLE_RICH_EMP, а также все ограничения целостности таблицы EMP и всех других представлений, определенных над этой базовой таблицей. В результате операция U1 будет отвергнута системой, а операция U2 будет «успешно» выполнена. Другими словами, повторится Случай 2.

Случай 4. В определении представления MIDDLE_RICH_EMP раздел WITH CHECK OPTION отсутствует, а в определении MORE_RICH_EMP содержится раздел WITH LOCAL CHECK OPTION.

Понятно, что в этом варианте операция U2 не сработает (ее выполнение не будет допущено условием «ограничения целостности» представления MORE_RICH_EMP). Но операция U1 (увеличение размера зарплаты служащих) будет успешно выполнена, поскольку она не противоречит локальным ограничениям представления MORE_RICH_EMP.

Случай 5. В определениях представлений MIDDLE_RICH_EMP и MORE_RICH_EMP содержится раздел WITH LOCAL CHECK OPTION.

Выполнение обеих операций U1 и U2 будет справедливо отвергнуто. На первый взгляд все в порядке. Но если над представлением MORE_RICH_EMP будет определено еще одно представление V, то мы можем получить ситуацию Случая 2, где V будет играть роль MORE_RICH_EMP, а MIDDLE_RICH_EMP – роль MORE_RICH_EMP.

Случай 6. В определении представления MIDDLE_RICH_EMP содержится раздел WITH CASCADED CHECK OPTION, а в определении MORE_RICH_EMP содержится раздел WITH LOCAL CHECK OPTION.

Снова, если над представлением MORE_RICH_EMP будет определено еще одно представление V, то мы можем попасть в ситуацию Случая 2, где V будет играть роль MORE_RICH_EMP, а MIDDLE_RICH_EMP – роль MORE_RICH_EMP.

Случай 7. В определении представления MIDDLE_RICH_EMP раздел WITH CHECK OPTION отсутствует, а в определении MORE_RICH_EMP содержится раздел WITH CASCADED CHECK OPTION.

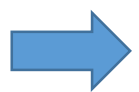
Если над представлением MORE_RICH_EMP будет определено еще одно представление V, то мы можем попасть в ситуацию Случая 3, где V будет играть роль MORE_RICH_EMP, а MIDDLE_RICH_EMP – роль MORE_RICH_EMP.

Случай 8. В определении представления MIDDLE_RICH_EMP содержится раздел WITH LOCAL CHECK OPTION, а в определении MORE_RICH_EMP – раздел WITH CASCADED CHECK OPTION.

Если над представлением MORE_RICH_EMP будет определено еще одно представление V, то мы можем получить ситуацию Случая 3, где V будет играть роль MORE_RICH_EMP, а MIDDLE_RICH_EMP – роль MORE_RICH_EMP.

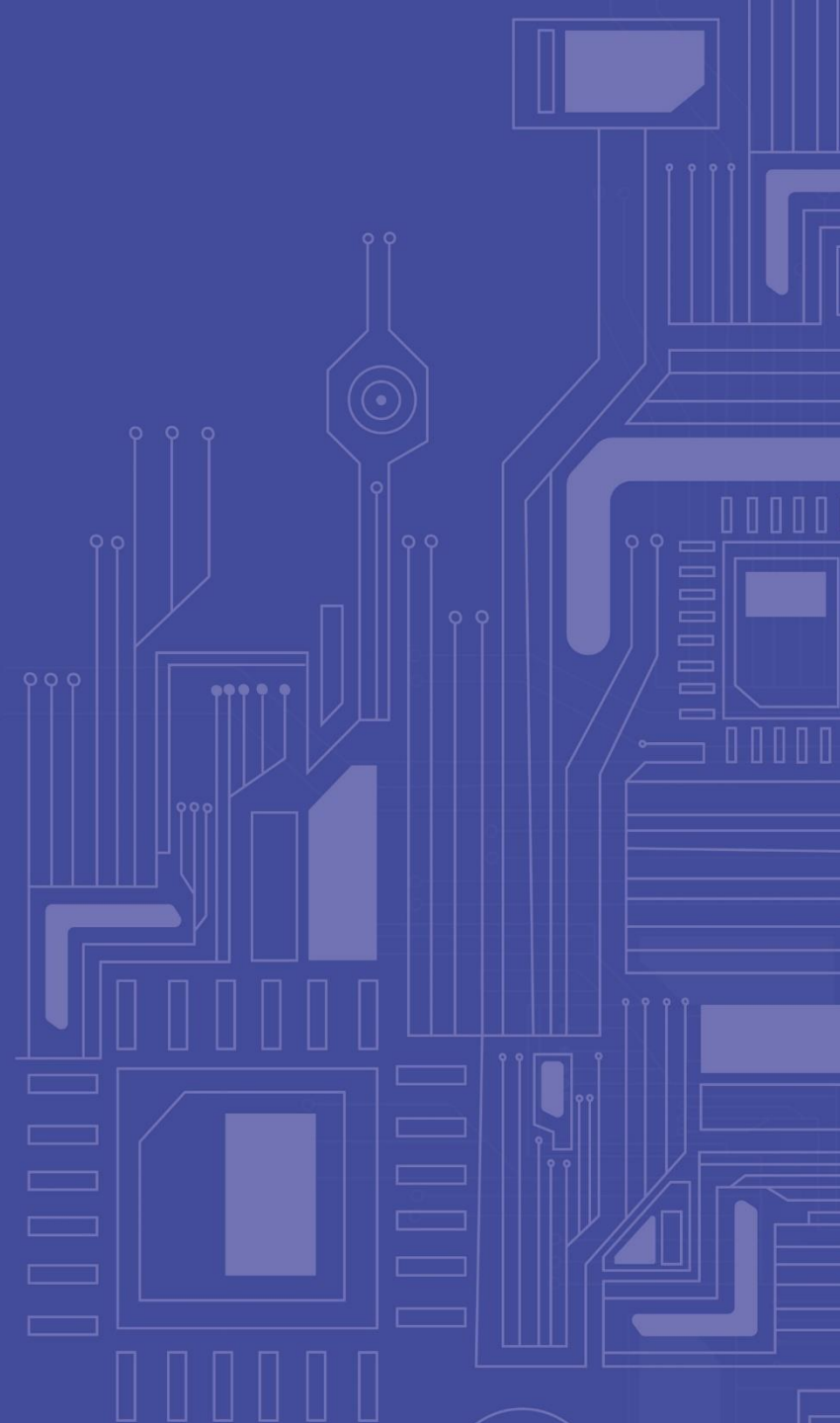
Случай 9. В определениях представлений MIDDLE_RICH_EMP и MORE_RICH_EMP содержится раздел WITH CASCADED CHECK OPTION.

Только в этом случае операции обновления будут выполняться корректно, независимо от того, имеются ли в базе данных представления, определенные над MORE_RICH_EMP или между MORE_RICH_EMP, MIDDLE_RICH_EMP и EMP.



Очевидный вывод из приведенного анализа заключается в том, что единственным способом обеспечить корректность выполнения операций обновления через представления (допускающие операции обновления) является включение в определение каждого представления раздела WITH CASCADED CHECK OPTION.

ТРИГГЕРЫ



Триггером называется хранимая в базе данных процедура, автоматически вызываемая СУБД при возникновении соответствующих условий.

SQL: 1999

В языке обеспечиваются возможности определения триггеров, которые вызываются («срабатывают») при вставке одной или нескольких строк в указанную таблицу, при модификации одной или нескольких строк в указанной таблице или при удалении одной или нескольких строк из указанной таблицы. Вообще говоря, триггер может производить любое действие, необходимое для соответствующего приложения. Можно определить триггеры, срабатывающие по одному разу для операций INSERT, UPDATE или DELETE, но существует и возможность определения триггеров, вызываемых при вставке, модификации или удалении каждой отдельной строки. Таблица, с которой связывается определение триггера, называется **предметной таблицей** (subject table), а оператор SQL, выполнение которого приводит к срабатыванию триггера, мы будем называть **инициирующим** (triggering SQL statement).

Триггеры могут срабатывать *после* и *до* реального выполнения инициирующего оператора SQL. В теле триггера допускается доступ к значениям вставляемых, модифицируемых и удаляемых строк. В случае операции модификации возможен доступ к значениям строк до модификации и к значениям после модификации. В соответствии со стандартом SQL:1999 любой триггер ассоциируется только с одной базовой таблицей. Не допускается определение триггеров над представлениями

Можно придумать различные способы полезного применения механизма триггеров, но принято считать, что основными областями использования этого механизма являются следующие.

- **Журнализация и аудит.** С помощью триггеров можно отслеживать изменения таблиц, для которых требуется поддержка повышенного уровня безопасности. Данные об изменении таблиц могут сохраняться в других таблицах и включать, например, идентификатор пользователя, от имени которого выполнялась операция обновления; временную метку операции обновления; сами обновляемые данные и т. д.
- **Согласование и очистка данных.** С любым простым оператором SQL, обновляющим некоторую таблицу, можно связать триггеры, производящие соответствующие обновления других таблиц. Например, с операцией вставки новой строки в таблицу EMP (прием на работу нового служащего) можно было связать триггер, модифицирующий значения столбцов DEPT_EMP_NO и DEPT_TOTAL_SAL строки таблицы DEPT со значением столбца DEPT_NO, которое соответствует номеру отдела нового служащего.
- **Операции, не связанные с изменением базы данных.** В триггерах могут выполняться не только операции обновления базы данных. Стандарт SQL позволяет определять хранимые процедуры (которые могут вызываться из триггеров), посылающие электронную почту, печатающие документы и т. д.

Синтаксис определения
триггера:

```
trigger_definition ::=
    CREATE TRIGGER trigger_name
    { BEFORE | AFTER }
    { INSERT | DELETE | UPDATE [ OF column_commalist ] }
    ON table_name [ REFERENCING
old_or_new_values_alias_list ]
    triggered_action
triggered_action ::=
    [ FOR EACH { ROW | STATEMENT } ]
    [ WHEN left_paren conditional_expression right_paren ]
    triggered_SQL_statement
triggered_SQL_statement ::= SQL_procedure_statement
    | BEGIN ATOMIC
        SQL_procedure_statement_semicolonlist
    END
old_or_new_values_alias ::= OLD [ ROW ] [ AS ] correlation_name
    | NEW [ ROW ] [ AS ] correlation_name
    | OLD TABLE [ AS ] identifier
    | NEW TABLE [ AS ] identifier
```

Синтаксис удаления триггера:

```
DROP TRIGGER trigger_name
```

Триггеры BEFORE и AFTER

Если в определении триггера указано ключевое слово BEFORE, то триггер будет срабатывать непосредственно до выполнения операции обновления базовой таблицы соответствующим инициирующим оператором SQL. При задании ключевого слова AFTER триггер будет вызываться немедленно после выполнения инициирующего оператора.

Триггеры INSERT, UPDATE и DELETE

При задании ключевого слова INSERT к срабатыванию триггера может привести только выполнение операции вставки строк в предметную таблицу. Если указываются ключевые слова UPDATE или DELETE, то число возможных событий, приводящих к срабатыванию триггера, возрастает. Кроме явных операций модификации строк предметной таблицы или удаления из нее строк к срабатыванию триггера могут привести ссылочные действия

Триггеры ROW и STATEMENT

Если в определении триггера присутствует конструкция FOR EACH ROW, то триггер будет вызываться для каждой строки предметной таблицы, обновляемой инициирующим SQL-оператором. Если же задано FOR EACH STATEMENT (или явная спецификация FOR EACH отсутствует), то триггер сработает один раз на всем протяжении процесса выполнения инициирующего SQL-оператора.

Раздел WHEN

Включение в определение триггера раздела WHEN с соответствующим условным выражением позволяет более точно специфицировать условие применимости триггера. Вычисление условного выражения производится над строками предметной таблицы, и триггер срабатывает только в том случае, когда значением условного выражения является true.

Операции, которые должны быть выполнены при срабатывании триггера, специфицируются в синтаксической конструкции `triggered_SQL_statement` (будем называть ее *инициируемым SQL-оператором*). Как видно из синтаксических правил, возможны два вида построения этой конструкции: в виде одиночного оператора SQL и в виде списка операторов со скобками `BEGIN ATOMIC` и `END`.

ПРИМЕР 1 Прием на работу нового служащего.

```
CREATE TRIGGER DEPT_CORRECTION AFTER INSERT ON EMP
FOR EACH ROW
WHEN (EMP.DEPT_NO IS NOT NULL)
UPDATE DEPT SET
DEPT_EMP_NO = DEPT_EMP_NO + 1,
DEPT_TOTAL_SAL = DEPT_TOTAL_SAL + EMP_SAL
WHERE DEPT.DEPT_NO = EMP.DEPT_NO;
```

ПРИМЕР 2 Увольнение служащего с сохранением информации о нем в отдельной таблице.

EMP_DISMISSED	
EMP_NO	: EMP_NO
EMP_NAME	: VARCHAR
DEPT_NO	: DEPT_NO

```
CREATE TRIGGER EMP_DISMISSION AFTER DELETE ON EMP
FOR EACH ROW
BEGIN ATOMIC
INSERT INTO EMP_DISMISSED
ROW (EMP.EMP_NO, EMP.EMP_NAME, EMP.DEPT_NO);
UPDATE DEPT SET
DEPT_EMP_NO = DEPT_EMP_NO - 1,
DEPT_TOTAL_SAL = DEPT_TOTAL_SAL - EMP_SAL
WHERE DEPT.DEPT_NO = EMP.DEPT_NO
END;
```

Понадобился составной оператор, потому что действия, определенные в теле триггера, затрагивают более одной таблицы.



При выполнении каждого триггера система устанавливает контекст выполнения триггера. Выполнение любого оператора SQL, обновляющего базовую таблицу базы данных, может привести к срабатыванию одного или нескольких триггеров, а выполнение операторов SQL, содержащихся в триггерах, может привести к обновлению других базовых таблиц. Эти «внутритриггерные» (инициируемые) операторы выполняются в контексте текущего триггера, но их выполнение может привести к срабатыванию других триггеров. Для каждого из «вторичных» триггеров образуется собственный контекст выполнения, позволяющий определить их действия точно и независимо от действий первого набора триггеров. Выполнение вторичных триггеров может привести к срабатыванию «третичных» триггеров и т.д. – допускается произвольная глубина вложенности. Для каждого триггера на каждом уровне образуется собственный контекст.



Контекст выполнения триггера всегда является атомарным, т.е. инициируемый SQL-оператор либо успешно завершается, либо результаты его действия гарантированно отсутствуют в базе данных.

ПРИМЕР С КОНТЕКСТОМ



Предположим, что в нашей базе данных EMP-DEPT-PRO должно поддерживаться правило, в соответствии с которым каждый служащий, становящийся руководителем проекта, автоматически получает прибавку к заработной плате в 10 000 руб. (Для простоты будем считать, что снятие служащего с должности руководителя проекта не приводит к автоматическому изменению его зарплаты и что для каждого служащего, являющегося руководителем проекта, определен номер отдела, в котором он работает.) Тогда мы могли бы определить триггер CHANGE_MNG_NO следующим образом:

```
CREATE TRIGGER CHANGE_MNG_NO AFTER UPDATE OF PRO_MNG ON PRO
FOR EACH ROW
  UPDATE EMP SET EMP_SAL = EMP_SAL + 10000.00
  WHERE EMP_NO = PRO_MNG;
```

```
CREATE TRIGGER DEPT_CORRECTION_1
AFTER UPDATE OF EMP_SAL ON EMP
REFERENCING OLD ROW AS OLD_EMP NEW ROW AS NEW_EMP
FOR EACH ROW
  UPDATE DEPT SET
    DEPT_TOTAL_SAL =
      DEPT_TOTAL_SAL + NEW_EMP.EMP_SAL -
      OLD_EMP.EMP_SAL
  WHERE EMP.DEPT_NO = DEPT.DEPT_NO;
```

Но очевидно, что для поддержания корректности данных в таблице DEPT нам требуется триггер, условием срабатывания которого было бы изменение значений столбца EMP_SAL в таблице EMP. Определим соответствующий триггер DEPT_CORRECTION_1:

Пусть теперь выполняется операция

```
UPDATE PRO SET PRO_MNG = 4455
WHERE PRO_NO = 554;
```

Сразу после выполнения этой операции сработает триггер CHANGE_MNG_NO. Этот триггер будет выполняться в контексте, который мы для удобства назовем контекстом CMN. Исходный оператор модификации в действительности изменяет только одну строку таблицы PRO,

но триггеру CHANGE_MNG_NO это неизвестно, и он будет работать так, как если бы изменялось произвольное число строк таблицы PRO. Выполнение операции модификации таблицы EMP приведет к срабатыванию триггера DEPT_CORRECTION_1. В этот момент контекст CMN будет «упрятан в стек», образуется и станет активным контекст следующего триггера – контекст DR1. После завершения выполнения этого триггера контекст DR1 больше не требуется, и он ликвидируется, а из стека восстанавливается контекст CMN, в котором и будет завершено выполнение триггера CHANGE_MNG_NO.

Контекст выполнения триггера служит для того, чтобы обеспечить СУБД данными, необходимыми для корректного выполнения иницилируемого оператора SQL. Эти данные представляют собой набор *изменений состояния*, где каждое изменение состояния описывает изменение данных в целевой таблице триггера.

Изменение состояния включает следующие данные:

- Триггерное событие – INSERT, UPDATE или DELETE;
- Имя предметной таблицы триггера;
- Имена столбцов предметной таблицы, специфицированных в определении триггера (только для триггеров по UPDATE);
- Набор *переходов* (представление всех строк, вставляемых в предметную таблицу, модифицируемых в ней или удаляемых из нее), список всех триггеров уровня STATEMENT, уже выполненных в некотором (не обязательно активном) контексте выполнения, и список всех триггеров уровня ROW, уже выполненных в некотором (не обязательно активном) контексте выполнения, и строк, над которыми эти триггеры выполнялись.

При создании контекста выполнения триггера его набор изменений состояния изначально пуст. В набор изменений состояния добавляется каждое встречающееся «новое» изменение состояния, в котором не дублируются триггерное событие существующего изменения состояния, имя предметной таблицы и имена столбцов предметной таблицы. Набор переходов каждого изменения состояния изначально пуст, и переходы добавляются при каждом обновлении предметной таблицы, ассоциированной с изменением состояния (включая обновления, производимые ссылочными действиями).

В определении триггера может присутствовать раздел `REFERENCING old_or_new_values_alias_list`, причем список определений псевдонимов может включать следующие элементы:

```
OLD [ ROW ] [ AS ] correlation_name  
NEW [ ROW ] [ AS ] correlation_name  
OLD TABLE [ AS ] identifier  
NEW TABLE [ AS ] identifier
```

- ➔ Каждая из этих конструкций может входить в список определений псевдонимов не более одного раза, и спецификации `OLD ROW` и `NEW ROW` могут присутствовать только в определении триггеров уровня `ROW`.
- ➔ Определяемые корреляционные имена и псевдонимы можно использовать внутри триггера для ссылок на значения предметной таблицы.
- ➔ Если определяется корреляционное имя для новых значений (`NEW ROW`) или псевдоним для нового содержимого таблицы (`NEW TABLE`), то эти имена можно использовать для ссылок на значения, которые будут существовать в предметной таблице *после* выполнения операций `INSERT` или `UPDATE`.
- ➔ Если же определяется корреляционное имя для старых значений (`OLD ROW`) или псевдоним для старого содержимого таблицы (`OLD TABLE`), то данные имена можно использовать для ссылок на значения, которые существовали в предметной таблице до выполнения операций `UPDATE` или `DELETE`.
- ➔ Конечно, нельзя использовать `NEW ROW` или `NEW TABLE` в триггерах `DELETE`, поскольку никакие новые значения не создаются. Аналогично, нельзя использовать `OLD ROW` или `OLD TABLE` в триггерах `INSERT`, поскольку никакие старые значения не существовали.

- ➔ Таблицы, на которые указывают корреляционные имена или псевдонимы, называются **переходными**. Эти таблицы не сохраняются в базе данных долговременно; они создаются и уничтожаются динамически, по мере надобности в контексте выполнения триггера. В триггерах уровня ROW можно использовать корреляционное имя, определенное в конструкции OLD ROW, для ссылки на значения строки, удаляемой или модифицируемой иницирующим оператором, в том виде, в котором данная строка существовала в предметной таблице до того, как была удалена или модифицирована при выполнении иницирующего оператора. В триггерах этого уровня можно также использовать псевдоним, определенный в конструкции OLD TABLE, для ссылки на любое значение переходной таблицы в том виде, в котором она находилась до удаления или модификации очередной строки при выполнении иницирующего оператора. Аналогично обстоят дела с использованием корреляционных имен и псевдонимов, определенных в конструкциях NEW ROW и NEW TABLE.
- ➔ Для триггеров категории BEFORE имеется существенное ограничение: в них не разрешается использовать конструкции OLD TABLE и NEW TABLE, а внутритриггерный SQL-оператор не может производить какие-либо изменения в базе данных. Основанием для такого ограничения является то, что на переходные таблицы, порождаемые OLD TABLE и NEW TABLE, могут существенно влиять ссылочные действия, которые активизируются в результате изменений базы данных при выполнении внутритриггерного SQL-оператора. Поэтому значения строк в таких таблицах могут оказаться нестабильными и недостаточно предсказуемыми, если триггер срабатывает раньше действия триггерного оператора SQL.

Обработка нескольких триггеров, связанных с одной предметной таблицей

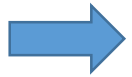
В **SQL:1999** не запрещается определение нескольких триггеров, ассоциированных с одной предметной таблицей, относящихся к одной и той же категории (BEFORE или AFTER) и срабатывающих по одному и тому же событию. Понятно, что при возникновении условия срабатывания всех таких триггеров система должна выбрать порядок, в котором они будут выполняться.

Решение, принятое в SQL, является предельно простым, хотя и несколько странным. При определении каждого триггера фиксируется временная метка выполнения оператора CREATE TRIGGER, и все триггеры, ассоциированные с одной предметной таблицей, относящиеся к одной и той же категории (BEFORE или AFTER) и срабатывающие по одному и тому же событию, упорядочиваются в соответствии со своими временными метками. Тогда при возникновении условия срабатывания всех триггеров одной группы сначала выполняется первый триггер, затем второй и т.д. В стандарте не специфицируется точность временной метки, связываемой с триггером, и если в одной группе обнаруживаются два или более триггеров с неразличимыми временными метками, то порядок их выполнения должен определяться в реализации.

Строже, чем атомарность триггеров

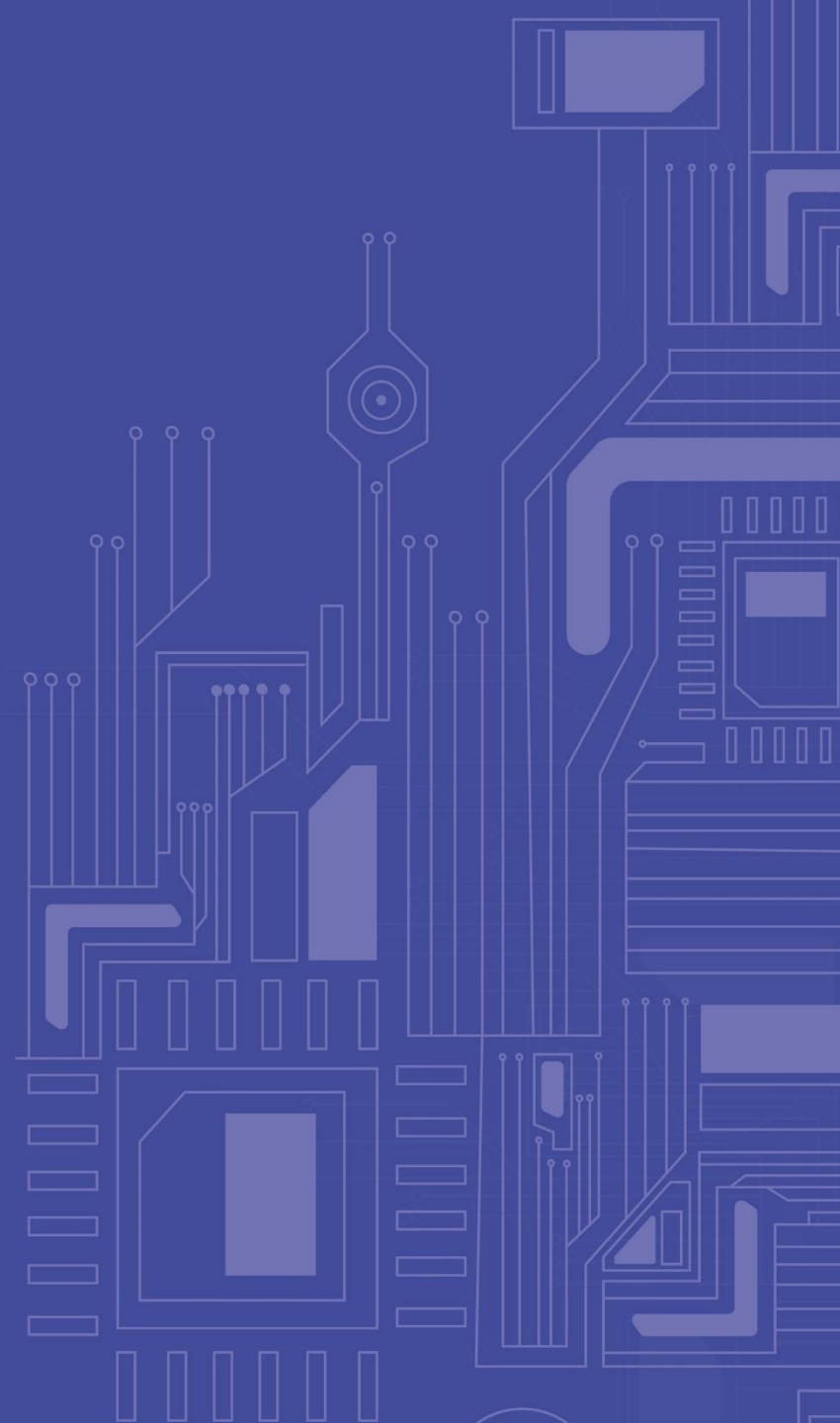
Неуспешное выполнение хотя бы одного триггера из группы с одинаковым условием срабатывания должно приводить к отмене результатов выполнения инициируемых SQL-операторов всех триггеров этой группы, а также к отмене результатов выполнения самого инициирующего SQL-оператора

Ссылочные действия автоматически модифицируют значения внешнего ключа соответствующей таблицы при удалении или модификации строк таблицы, на которую указывают ссылки.



В SQL:1999 декларируется, что контроль всех ограничений целостности (включая ссылочные ограничения) и выполнение всех ссылочных действий должны производиться до срабатывания триггеров категории AFTER. Если выполняется некоторая операция обновления таблицы T, то после ее выполнения и срабатывания всех ссылочных действий инициируются все триггеры, ассоциированные с таблицей T и видом произведенной операции, а также соответствующие триггеры, ассоциированные с любой таблицей, которая затрагивалась ссылочным действием, если в этой таблице была изменена хотя бы одна строка. Конечно, срабатывание триггера может привести к новым ссылочным действиям, которые повлекут за собой срабатывание других триггеров и т. д.

ПОДДЕРЖКА АВТОРИЗАЦИИ ДОСТУПОВ К ДАННЫМ В SQL



АВТОРИЗАЦИЯ ДОСТУПА

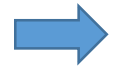
Метод *авторизации доступа*, используемый в SQL, относится к *мандатным (mandatory) видам защиты данных*. При этом подходе с каждым зарегистрированным в системе пользователем (*субъектом*) и каждым защищаемым объектом системы связывается *мандат*, определяющий действия, которые может выполнять данный субъект над данным объектом. В отличие от такого подхода, при применении *дискреционного (discretionary) метода ограничения доступа* с каждым из объектов системы связывается одна или несколько *категорий пользователей*, каждой из которых позволяются или запрещаются некоторые действия над объектом.

В совокупности в SQL:1999 может поддерживаться девять видов защиты разных объектов в соответствии со следующими возможными действиями:

Вид защиты и соответствующее действие	Название привилегии	Применимо к следующим объектам
Просмотр	SELECT	Таблицы, столбцы, подпрограммы, вызываемые из SQL
Вставка	INSERT	Таблицы, столбцы
Модификация	UPDATE	Таблицы, столбцы
Удаление	DELETE	Таблицы
Ссылка	REFERENCES	Таблицы, столбцы
Использование	USAGE	Домены, определенные пользователями типы, наборы символов, порядки сортировки символов, преобразования
Иниципирование	TRIGGER	Таблицы
Выполнение	EXECUTE	Подпрограммы, вызываемые из SQL
Подтипизация	UNDER	Структурные типы



В SQL-ориентированной системе каждому зарегистрированному в системе пользователю соответствует его уникальный идентификатор (в стандарте используется термин *идентификатор авторизации*, *authorization identifier* – *authID*).



В SQL:1999 не зафиксированы точные правила представления идентификатора пользователя, хотя обычно в реализациях SQL *ниладическая* функция CURRENT USER выдает текстовую строку, содержащую регистрационное имя пользователя, как оно сохраняется в файлах соответствующей операционной системы (ОС).



Привилегии доступа к объектам базы данных могут предоставляться пользователям, представляемым своими идентификаторами, а также ролям, которые, в свою очередь, могут предоставляться пользователям.



Кроме того, в SQL поддерживается концепция *псевдоидентификатора* (или *идентификатора псевдо*) *пользователя* PUBLIC, который соответствует любому приложению или пользователю, зарегистрированному в системе баз данных. «Пользователю» PUBLIC могут предоставляться привилегии доступа к объектам базы данных, как и любому другому пользователю.



При разработке средств контроля доступа к объектам баз данных создатели SQL придерживались принципа сокрытия информации об объектах, содержащихся в схеме базы данных, от пользователей, которые лишены доступа к этим объектам. Другими словами, если некоторый пользователь не обладает, например, привилегией на просмотр таблицы PRO, то при выполнении операции SELECT * FROM PRO он получит такое же диагностическое сообщение, как если бы таблица PRO не существовала. Если бы в случае отсутствия этой таблицы и в случае отсутствия привилегии доступа выдавались разные диагностические сообщения, то непривилегированный пользователь получил бы данные о том, что интересующая его таблица существует, но он лишен доступа к ней.

В модели контроля доступа SQL создатель любого объекта базы данных автоматически становится владельцем этого объекта. При этом владелец объекта может идентифицироваться либо своим идентификатором пользователя, либо именем своей роли. Вообще говоря, владелец объекта обладает полным набором привилегий для выполнения действий над объектом (с одним исключением, которое мы обсудим в данном разделе позже). Владелец объекта, помимо прочего, обладает привилегией на передачу всех (или части) своих привилегий другим пользователям или ролям. В частности, владелец объекта может передать другим пользователям или ролям привилегию на передачу привилегий последующим пользователям или ролям (эти действия с передачей привилегии на передачу привилегий могут продолжаться рекурсивно).

Во многих реализациях поддерживаются привилегии уровня DBA (DataBase Administrator) для возможности выполнения операций DDL – Data Definition Language (CREATE, ALTER и DROP над объектами, входящими в схему базы данных). В стандарте SQL требуется лишь соблюдение следующих правил.

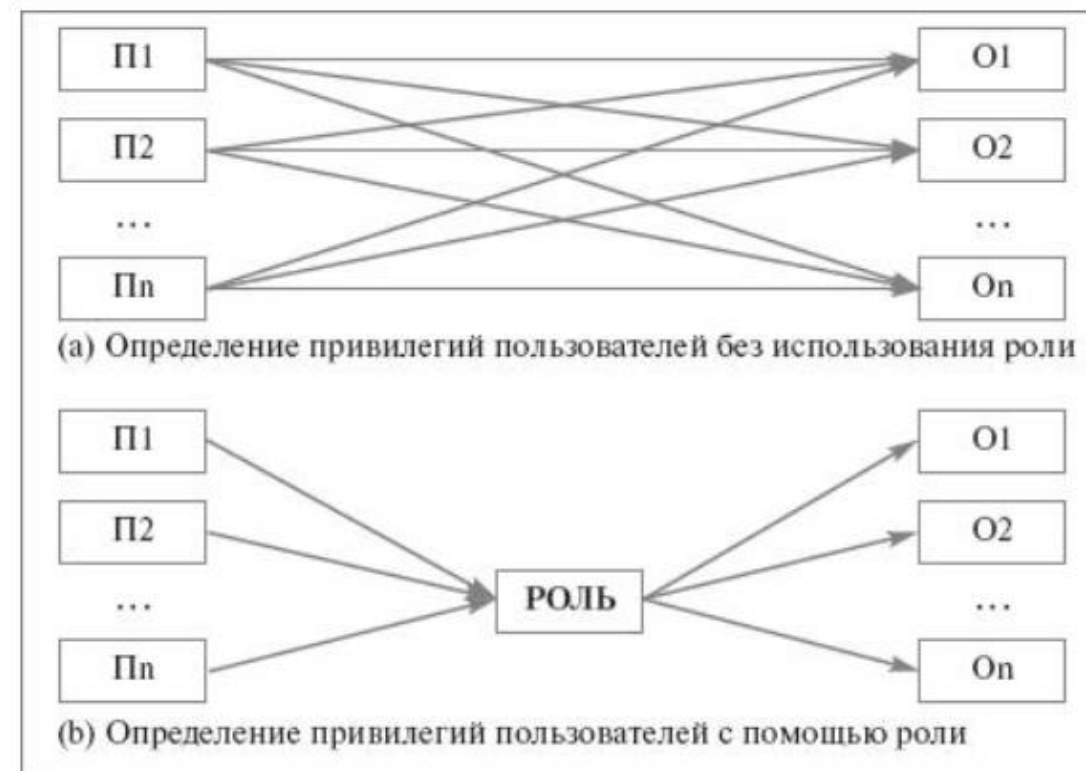
- Любые пользователь или его роль могут выполнять любые операции DDL внутри схемы, которой владеют¹⁷¹⁾.
- Не допускается выполнение каких-либо операций DDL внутри схемы, которой не владеет пользователь или роль, пытающиеся выполнить соответствующую операцию.
- Эти правила не допускают исключений.

Любой пользователь характеризуется своим идентификатором авторизации (authID). Идентификатор авторизации может являться либо *идентификатором пользователя*, либо *именем роли*.

В соответствии с правилами стандарта SQL, все authID должны отслеживаться СУБД (имеются в виду все authID, для которых существует хотя бы одна привилегия). И в стандарте поддерживаются точные правила порождения и распространения привилегий. Привилегии по отношению к объекту базы данных предоставляются системой владельцу схемы при создании объекта в этой схеме, и привилегии могут явно передаваться от имени одного authID другому authID при наличии у первого authID привилегии на передачу привилегий.

Каждая стрелка на рисунке соответствует мандату доступа (паре $\langle \text{authID}, \text{набор_привилегий_доступа_к_объекту_БД} \rangle$), который требуется сохранять в каталоге базы данных и проверять при попытке доступа от имени authID.

Как видно, в случае (а) требуется сохранение и проверка $n*m$ мандатов, где n – число пользователей в группе, а m – число объектов базы данных, для которых пользователи группы должны иметь одни и те же привилегии. В случае (б) число требуемых для корректной работы мандатов равно лишь $n+m$, и схема авторизации резко упрощается.



В любой момент заданная SQL-сессия ассоциируется с идентификатором пользователя, называемым идентификатором пользователя SQL-сессии и с именем роли, называемым именем роли SQL-сессии.

Если сессия образуется с помощью оператора CONNECT , то authID указывается в качестве параметра соответствующей операции. Для реализаций SQL допускается, чтобы пользовательский идентификатор SQL-сессии совпадал с регистрационным именем пользователя с точки зрения операционной системы или являлся идентификатором, специально устанавливаемым специалистами организации, ответственными за обеспечение безопасности.

Кроме того, допускается наличие в реализации оператора SET SESSION AUTHORIZATION, применение которого приводит к смене пользовательского идентификатора SQL-сессии. В начале SQL-сессии значение текущего идентификатора пользователя SQL-сессии совпадает со значением пользовательского идентификатора SQL-сессии, и такая ситуация сохраняется до тех пор, пока пользовательский идентификатор SQL-сессии не будет каким-либо образом изменен. Значение текущего пользовательского идентификатора SQL-сессии возвращается вызовом *ниладической* функции SESSION_USER.

Для каждой SQL-сессии существует также текущее имя роли (это имя можно получить путем вызова функции CURRENT_ROLE).

Для создания новой роли используется оператор `CREATE ROLE`, определяемый следующим синтаксическим правилом:

```
CREATE ROLE role_name  
[ WITH ADMIN { CURRENT_USER | CURRENT_ROLE } ]
```

Имя создаваемой роли должно отличаться от любого идентификатора авторизации, уже определенного и сохраненного в базе данных. В случае успешного создания роли некоторый `authID` получает привилегию на исполнение данной роли. Если в операторе `CREATE ROLE` не содержится раздел `WITH ADMIN`, то привилегию на исполнение роли получает текущий идентификатор пользователя SQL-сессии, если значение этого идентификатора отлично от `NULL`; иначе привилегия на исполнение роли дается текущему имени роли сессии.

Если в состав оператора включается раздел `WITH ADMIN`, то можно выбрать, будет ли являться владельцем роли `authID`, соответствующий текущему идентификатору пользователя SQL-сессии, или `authID`, соответствующий текущему имени роли (при условии, что соответствующие текущий идентификатор или текущее имя не содержат `NULL`). Кроме того, включение этого раздела означает, что `authID`-владелец роли получает право на передачу привилегии исполнения данной роли другим `authID`.

В соответствии со стандартом SQL:1999, привилегии, требуемые для выполнения оператора `CREATE ROLE`, определяются в реализациях SQL. Например, в некоторых реализациях выполнение этой операции разрешается только администратору базы данных.

Существующую роль можно ликвидировать с помощью оператора

```
DROP ROLE role_name
```

```
GRANT { ALL PRIVILEGES | privilege_commalist }  
  ON privilege_object  
TO { PUBLIC | authID_commalist } [ WITH GRANT OPTION ]  
  [ GRANTED BY { CURRENT_USER | CURRENT_ROLE } ]  
privilege ::= SELECT [ column_name_commalist ]  
             | DELETE  
             | INSERT [ column_name_commalist ]  
             | UPDATE [ column_name_commalist ]  
             | REFERENCES [ column_name_commalist ]  
             | USAGE  
             | TRIGGER  
             | EXECUTE  
privilege_object ::= [ TABLE ] table_name  
                    | DOMAIN domain_name  
                    | CHARACTER SET character_set_name  
                    | COLLATION collation_name  
                    | TRANSLATION translation_name
```

Поскольку authID может являться идентификатором пользователя или именем роли, привилегии могут передаваться от пользователей пользователям, от пользователей ролям, от ролей ролям и от ролей пользователям.

Включение в оператор необязательного раздела WITH GRANT OPTION означает, что получателям передаваемых привилегий дается также привилегия на дальнейшую передачу полученных привилегий, включая привилегию на передачу привилегий. Включение в оператор раздела GRANTED BY позволяет явно указать, передаются ли привилегии от имени текущего идентификатора пользователя или же текущего имени роли.

Включение в оператор необязательного раздела WITH GRANT OPTION означает, что получателям передаваемых привилегий дается также привилегия на дальнейшую передачу полученных привилегий, включая привилегию на передачу привилегий. Включение в оператор раздела GRANTED BY позволяет явно указать, передаются ли привилегии от имени текущего идентификатора пользователя или же текущего имени роли.

Поскольку создаваемое представление всегда основывается на одной или нескольких базовых таблицах (или представлениях), привилегии, которые получает создатель представления, должны основываться на привилегиях, которыми располагает текущий authID по отношению к этим базовым таблицам или представлениям.

Что происходит при смене привилегий владельца представления по отношению к таблицам, на которых основано это представление. Для простоты предположим, что представление V основано на базовой таблице T. Если во время создания V текущий authID (будущий владелец представления) обладал по отношению к T привилегиями SELECT и INSERT, то он будет обладать этими привилегиями и по отношению к V. Если впоследствии владелец представления получит по отношению к T дополнительные привилегии, то он (и все authID, которым передавались все привилегии – ALL PRIVILEGES для V) получит те же привилегии для V.

Передача ролей

```
GRANT role_name_commlist  
TO { PUBLIC | authID_commlist } [ WITH ADMIN OPTION ]  
[ GRANTED BY { CURRENT_USER | CURRENT_ROLE } ]
```

Оператор позволяет передавать произвольное число ролей произвольному числу authID (которые могут представлять собой идентификаторы пользователей или имена ролей). Как и в случае передачи привилегий, от данного authID можно передавать только те роли, которые были получены этим authID с привилегией на дальнейшую передачу (WITH ADMIN OPTION). При включении в состав оператора GRANT раздела GRANTED BY можно явно указать, что роли передаются от имени текущего идентификатора пользователя или же текущего имени роли.

ИЗМЕНЕНИЯ ТЕКУЩИХ ИДЕНТИФИКАТОРОВ ПОЛЬЗОВАТЕЛЕЙ И ИМЕН РОЛЕЙ



Оператор SET SESSION AUTHORIZATION

Для изменения текущего идентификатора пользователя SQL-сессии может использоваться оператор

SET SESSION AUTHORIZATION value_specification

Параметр value_specification может быть либо литералом (в данном случае литералом типа символьных строк), либо вызовом *ниладической* функции, такой, как CURRENT_USER, SESSION_USER и т. д. Если указанная спецификация значения не соответствует требованиям, предъявляемым в реализации к представлению идентификатора пользователя, или такого пользователя нет, операция изменения текущего идентификатора пользователя аварийно завершается. Внутри транзакции менять пользователя нельзя.

Оператор SET ROLE

Для смены текущего имени роли SQL-сессии можно использовать оператор

SET ROLE { value_specification | NONE }

Ограничения на выполнение операции SET ROLE почти совпадают с определенными в стандарте ограничениями на выполнение операции SET SESSION AUTHORIZATION. Наиболее важные отличия состоят в том, что эту операцию от имени текущего authID сессии всегда разрешается выполнять для ролей, которые переданы «пользователю» PUBLIC или данному текущему authID, а также в том, что всегда разрешается применение конструкции SET ROLE NONE (что значение текущего имени роли сессии становится неопределенным).

АННУЛИРОВАНИЕ ПРИВИЛЕГИЙ (1/2)

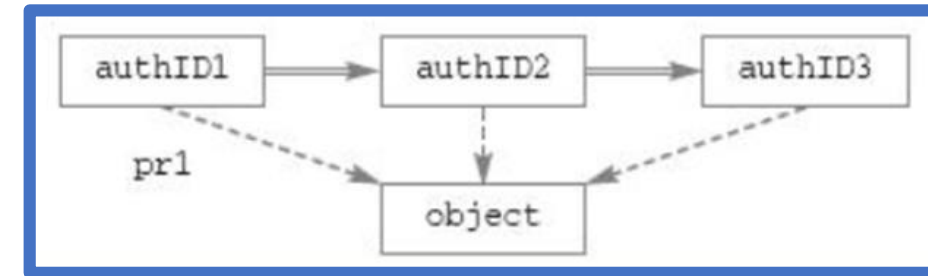


Если от имени некоторого authID некоторые привилегии или роли были переданы одному или нескольким другим authID, то впоследствии первый authID (в сессии, где этот authID является текущим) можно изъять, или *аннулировать*, переданные привилегии или роли путем применения оператора REVOKE.

```
REVOKE [ GRANT OPTION FOR] privilege_commalist
      ON privilege_object
FROM { PUBLIC | authID_commalist }
     [ GRANTED BY { CURRENT_USER | CURRENT_ROLE } ]
     { RESTRICT | CASCADE }
```

Важной особенностью оператора аннулирования привилегий является обязательность указания одного из ключевых слов RESTRICT или CASCADE.

Если в операторе содержится RESTRICT, то при выполнении операции система проверит, не передавалась ли какая-либо из указанных привилегий каким-либо authID от того authID, у которого привилегия должна быть аннулирована (это вполне возможно, если ранее привилегия была передана с правом передачи). Если это действительно так, операция не выполняется; в противном случае указанные привилегии у указанных authID аннулируются.



ПРИМЕР На рисунке authID1 является владельцем объекта object. Пунктирной стрелкой обозначена одна из привилегий pr1. От имени authID1 привилегия pr1 была передана authID2 вместе с привилегией на ее дальнейшую передачу. Наконец, от имени authID2 привилегия pr1 была передана authID3. Тогда операция аннулирования этой привилегии от имени authID1 у authID2 при наличии ключевого слова RESTRICT не будет выполнена успешно.

АННУЛИРОВАНИЕ ПРИВИЛЕГИЙ (2/2)



В том же примере привилегия была бы аннулирована для authID2 (и для authID3), если бы в операторе GRANT присутствовало ключевое слово CASCADE. В общем случае если выполняется операция REVOKE ... CASCADE, то указанные привилегии аннулируются у всех authID, прямо или косвенно (через промежуточные authID) получивших привилегии от текущего authID SQL-сессии, в которой выполняется данная операция.



Если в операторе содержится раздел GRANT OPTION FOR, но имеется ключевое слово RESTRICT, то указанные привилегии для указанных authID не аннулируются, но у указанных authID аннулируется привилегия передачи данных привилегий). Однако если в операторе одновременно содержатся и GRANT OPTION FOR, и CASCADE, то указанные привилегии аннулируются у всех authID, которые прямо или косвенно (через промежуточные authID) получили привилегии от текущего authID SQL-сессии, в которой выполняется данная операция.



Задание в операторе необязательного раздела GRANTED BY позволяет явно указать, что должно использоваться в качестве текущего authID – текущий пользовательский идентификатор или текущее имя роли SQL-сессии. Если раздел GRANTED BY в операторе REVOKE не содержится, то действия производятся от имени текущего authID SQL-сессии.



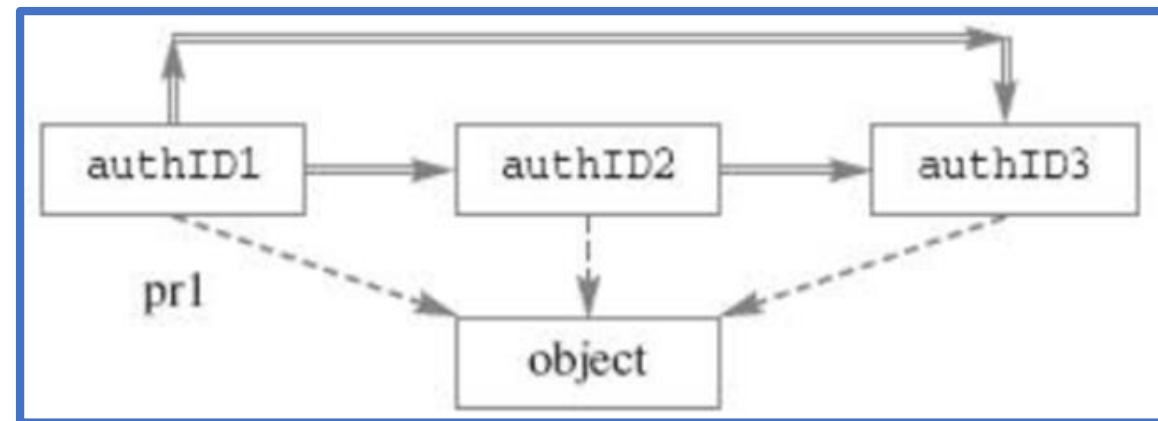
Если текущий (или указанный) authID не обладает ни одной из указанных в операторе REVOKE привилегий, то выполнение операции не производится (фиксируется ошибка). Если authID обладает некоторыми, но не всеми, привилегиями из числа указанных, то операция выполняется по отношению к этим некоторым привилегиям, но выдается предупреждение.

ПРИМЕР НА АННУЛИРОВАНИЕ ПРИВИЛЕГИЙ



ПРИМЕР Возможны ситуации, когда у некоторого **authID** остается некоторая привилегия после выполнения операции аннулирования у этого **authID** этой привилегии.

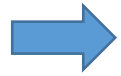
Здесь привилегия **pr1** передана от **authID1** к **authID2** вместе с правом на дальнейшую передачу этой привилегии. Далее, привилегия **pr1** передается от **authID2** к **authID3**.



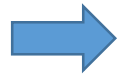
И затем выполняется прямая передача привилегии от **authID1** к **authID3** (на самом деле, порядок таких действий не является существенным). Теперь предположим, что от имени **authID1** выполняется операция

REVOKE pr1 ON object FROM authID2 CASCADED

В соответствии с правилами SQL:1999 после выполнения этой операции **authID3** будет продолжать владеть привилегией **pr1** по отношению к объекту **object**, поскольку получил данную привилегию *двумя разными способами*. Грубо говоря, операция **REVOKE**, выполняемая от имени **authID1**, выполняется только по тем путям графа идентификаторов авторизации и объектов базы данных, которые начинаются с узлов, соответствующих **authID**, указанных в разделе **FROM** этой операции.



Если при передаче от `authID1` к `authID2` привилегии на выполнение некоторых действий над некоторой таблицей `T` (например, `UPDATE`) явно не указывается список имен столбцов этой таблицы, то привилегия распространяется на все столбцы этой таблицы (включая столбцы, которые, возможно, еще будут созданы). Если действительно использовался такой способ передачи привилегий, то в дальнейшем можно аннулировать привилегию `authID2` на модификацию отдельных (уже определенных) столбцов таблицы `T`, оставив привилегию на модификацию всех остальных столбцов (включая те, которые еще не созданы).

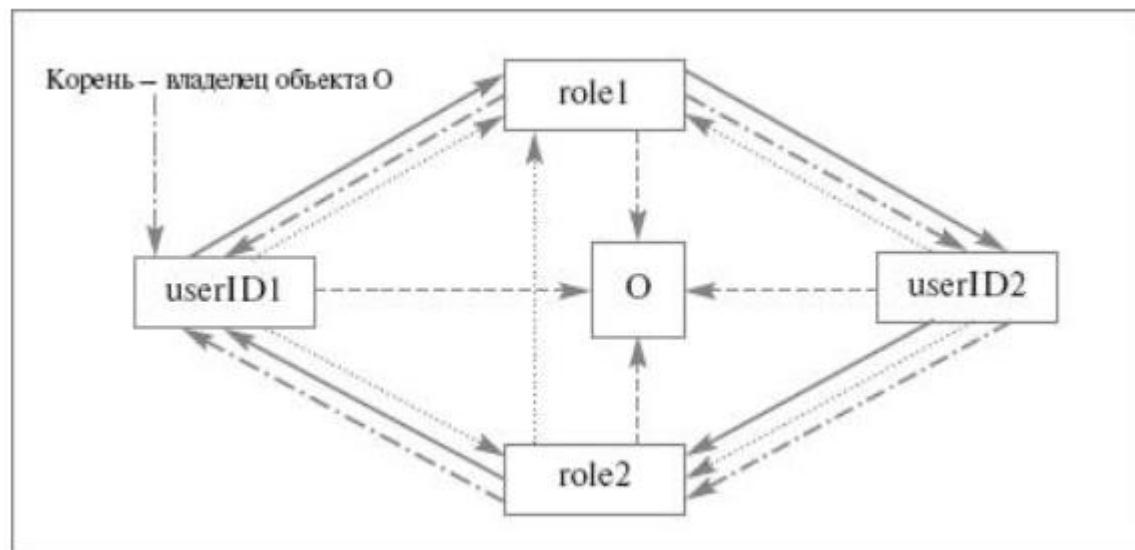


И последнее замечание. Если некоторая привилегия была передана **псевдоauthID PUBLIC**, то, конечно, этой привилегией обладают все **authID**. Но нет возможности аннулировать такую привилегию у отдельно указываемого **authID**. Привилегия была передана всем, и аннулировать ее можно только сразу у всех.

Действие операции аннулирования ролей очень похоже на действие операции аннулирования привилегий. Отличие состоит в том, что аннулируются не привилегии, а роли, а также в том, что для аннулирования привилегии на передачу роли используется раздел ADMIN OPTION FOR.

```
REVOKE [ ADMIN OPTION FOR ] role_name_commalist  
FROM { PUBLIC | authID_commalist }  
      [ GRANTED BY { CURRENT_USER | CURRENT_ROLE } ]  
      { RESTRICT | CASCADE }
```

СЛОЖНЫЙ CASE С РОЛЯМИ



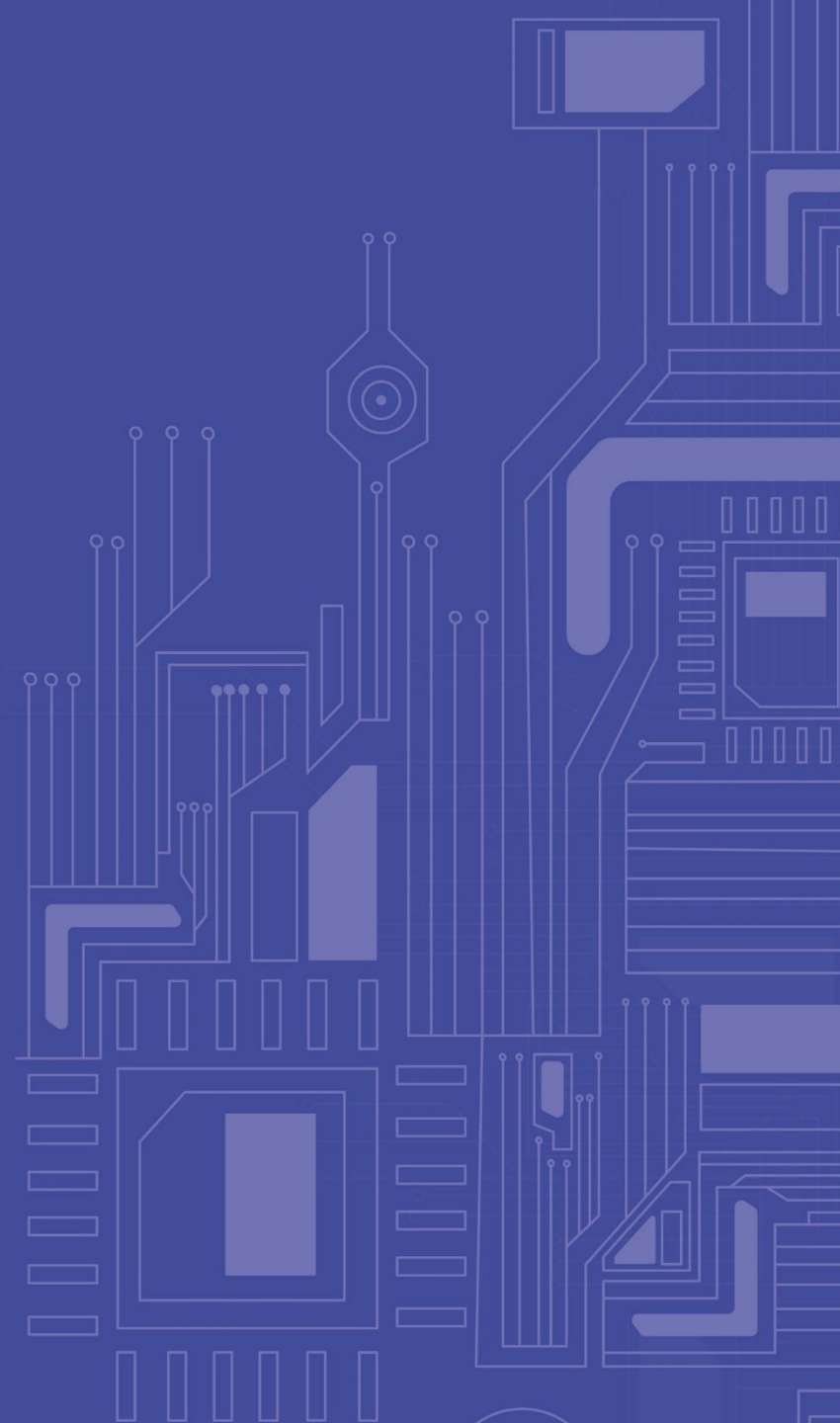
В любой новой СУБД пробуем воспроизвести этот кейс, и смотрим, что получится.

Как мог появиться такой граф? Пользователь с `authID`, равным `userID1` (это мы предположили для упрощения, а вообще-то это могло быть и именем роли), создает объект `o`, становится его владельцем и тем самым обладателем привилегии `pr` по отношению к этому объекту. Пользователь `userID1` предоставляет полномочие `pr` роли `role1` (с правом передачи). Затем пользователю `userID1` предоставляется роль `role1` (с правом передачи), и он получает право исполнять эту роль. От имени роли `role1` полномочие `pr` передается пользователю `userID2` (с правом передачи), и этот же пользователь получает право исполнять роль `role1` (с правом передачи). Пользователь `userID2` передает роли `role2` роль `role1` и полномочие `pr` (с правом передачи). Наконец, от имени роли `role2` полномочие `pr` и сама роль `role2` передаются пользователю `userID1`.

Попробуйте теперь проследить, как будет выполняться операция

```
REVOKE pr ON o FROM role1 CASCADED
```

УПРАВЛЕНИЕ ТРАНЗАКЦИЯМИ В SQL



Классическое понимание транзакции, характеризуемое аббревиатурой ACID (от Atomicity, Consistency, Isolation и Durability) подразумевает последовательность операций (например, над базой данных), обладающая следующими свойствами.

- **Атомарность** (Atomicity). Это свойство означает, что результаты всех операций, успешно выполненных в пределах транзакции, должны быть отражены в состоянии базы данных, либо в состоянии базы данных не должно быть отражено действие ни одной операции (конечно, здесь речь идет об операциях, изменяющих состояние базы данных). Свойство атомарности, которое часто называют свойством «все или ничего», позволяет относиться к транзакции, как к динамически образуемой составной операции над базой данных.
- **Согласованность** (Consistency). В классическом смысле это свойство означает, что транзакция может быть успешно завершена с *фиксацией* результатов своих операций только в том случае, когда действия операций не нарушают целостность базы данных, т. е. удовлетворяют набору ограничений целостности, определенных для этой базы данных. В стандарте SQL:1999 это свойство расширяется тем, что во время выполнения транзакции разрешается устанавливать точки согласованности (см. ниже про точки сохранения) и явным образом проверять ограничения целостности.
- **Изоляция** (Isolation). Требуется, чтобы две одновременно выполняемые транзакции никоим образом не действовали одна на другую. Другими словами, результаты выполнения операций транзакции T1 не должны быть видны никакой другой транзакции T2 до тех пор, пока транзакция T1 не завершится успешным образом.
- **Долговечность** (Durability). После успешного завершения транзакции все изменения, которые были внесены в состояние базы данных операциями этой транзакции, должны гарантированно сохраняться, даже в случае сбоя аппаратуры или программного обеспечения.

В соответствии со стандартом языка SQL:1999 транзакции могут образовываться явным образом с использованием оператора `START TRANSACTION`, либо неявно, когда выполняется оператор, для которого требуется контекст транзакции, а этого контекста не существует. Например, операторы `SELECT`, `UPDATE` или `CREATE TABLE` могут выполняться только в контексте транзакции, а для выполнения оператора `CONNECT` такой контекст не требуется, и выполнение оператора `CONNECT` не приводит к неявному образованию транзакции. Для завершения транзакции должен быть явно использован один из двух операторов – `COMMIT` (требование завершить транзакцию с фиксацией ее результатов) или `ROLLBACK` (требование завершить транзакцию с удалением результатов всех выполненных операций из состояния базы данных).



У каждой выполняемой транзакции имеются три характеристики, значения которых существенно влияют на действия системы при управлении транзакцией:

1. *уровень изоляции (isolation level),*
2. *режим доступа (access mode),*
3. *размер области диагностики.*

```
SET TRANSACTION mode_commlist
mode ::= isolation_level
        | access_mode
        | diagnostics_size
isolation_level ::= READ UNCOMMITTED
                  | READ COMMITTED
                  | REPEATABLE READ
                  | SERIALIZABLE
access_mode ::= READ ONLY
              | READ WRITE
diagnostics_size ::= DIAGNOSTIC SIZE value_specification
```

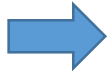
При неявном образовании транзакции эти характеристики устанавливаются по умолчанию: транзакция получает максимальный уровень изоляции от одновременно выполняемых транзакций; режим доступа, позволяющий выполнять и операции выборки, и операции обновления базы данных; и назначаемый по умолчанию размер области диагностики.

-  Операцию установки характеристик транзакции нельзя выполнять в контексте какой-либо активной транзакции. Выполнение операции допустимо только до образования первой транзакции SQL-сессии или между последовательно выполняемыми транзакциями этой сессии. В одном операторе SET TRANSACTION можно задать только по одному значению каждой из трех характеристик, но допускается последовательное выполнение нескольких таких операций с разными операндами.
-  Как видно из синтаксических правил, у характеристики режим доступа может быть указано одно из двух значений – READ ONLY или READ WRITE. Если устанавливается режим READ ONLY, то в транзакции нельзя будет выполнять никакие операции, изменяющие базу данных, в том числе операции обновления таблиц и определения новых объектов базы данных. Если режим доступа явно не указывается, по умолчанию принимается характеристика READ WRITE, если только в качестве значения характеристики уровень изоляции не указывается READ UNCOMMITTED (в этом случае устанавливается режим доступа READ ONLY).
-  Если указывается размер области диагностики, то после ключевых слов DIAGNOSTIC SIZE должен следовать целочисленный литерал, определяющий число диагностических элементов, которые должны разместиться в области диагностики (число исключительных ситуаций, предупреждений, сообщений об отсутствии данных и об успешном выполнении, которые будут вырабатываться при выполнении операторов внутри будущей транзакции).

Для явного образования транзакции поддерживается оператор `START TRANSACTION`, определяемый следующими синтаксическими правилами:

`START TRANSACTION mode_commalist`

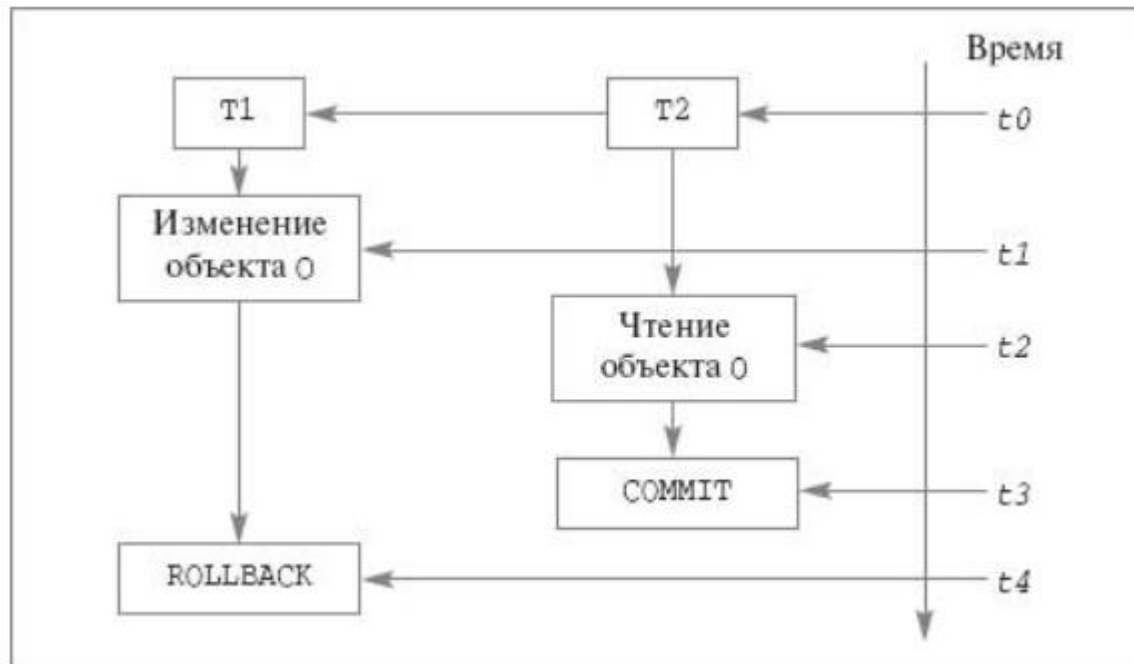
Этот оператор очень похож на `SET TRANSACTION`. Единственное (хотя и очень существенное) отличие состоит в том, что выполнение оператора `START TRANSACTION` приводит не только к установке характеристик транзакции, но и к реальной инициации транзакции.



Еще одна интересная деталь оператора установки характеристик транзакции состоит в том, что выполнение каждого следующего оператора `SET TRANSACTION` полностью перекрывает эффект выполнения предыдущего такого оператора. В частности, если в предыдущем операторе явно задавалось значение некоторой характеристики, а в следующем это значение принимается по умолчанию, то именно значение по умолчанию будет являться значением характеристики транзакции.

ФЕНОМЕН ГРЯЗНОГО ЧТЕНИЯ

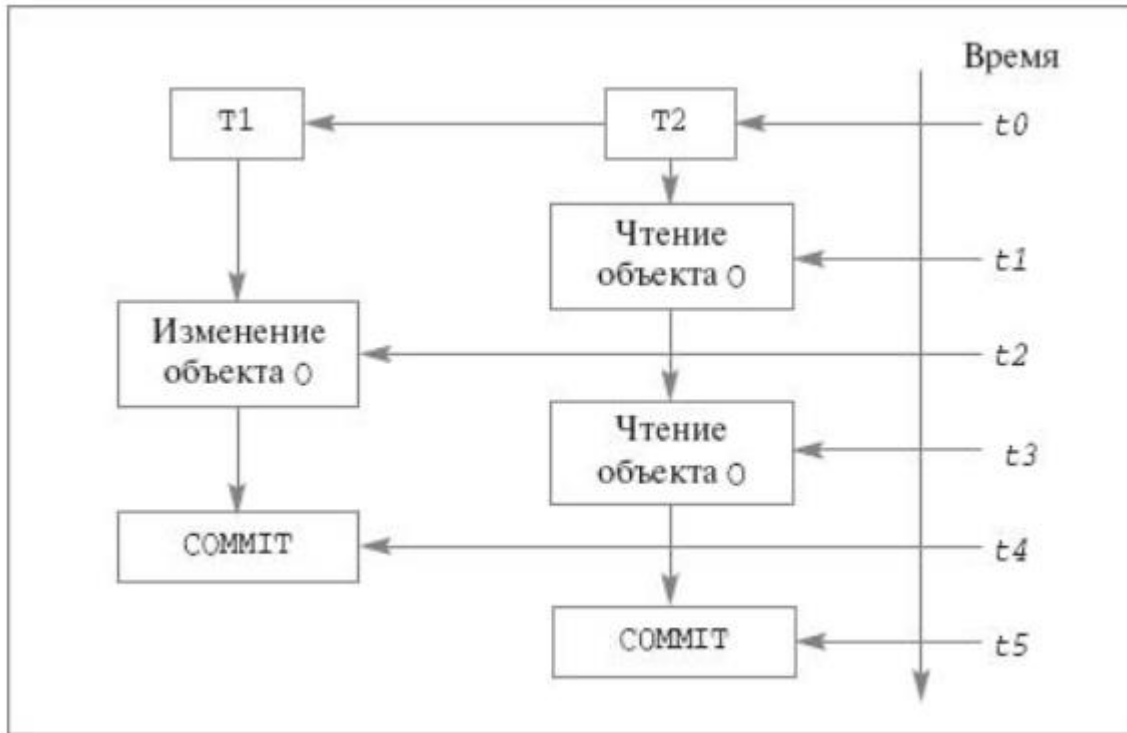
В SQL феномен «грязного» чтения может наблюдаться у транзакций, выполняемых на уровне изоляции **READ UNCOMMITTED**. Рекомендуется использовать этот уровень изоляции только в тех транзакциях, для выполнения функций которых точные данные не обязательны (например, в транзакциях, производящих статистическую обработку).



На этом рисунке показано, что в момент времени t_0 были образованы две транзакции T1 и T2. В момент времени t_1 транзакция T1 успешно выполняет операцию модификации некоторого объекта базы данных O. В момент времени t_2 ($t_2 > t_1$) транзакция T2 читает объект O, после чего успешно завершается в момент времени t_3 . Транзакция же T1 завершается в момент времени t_4 ($t_4 > t_3$), причем в ней выполняется оператор ROLLBACK, что приводит к ликвидации в базе данных последствий изменения объекта O. В результате оказывается, что в транзакции T2 обрабатывались данные, которые реально не существуют в базе данных (отсюда и термин «грязные» данные).

ФЕНОМЕН UNREPEATABLE READ

В SQL феномен неповторяемого чтения может наблюдаться у транзакций, выполняемых на уровне изоляции **READ COMMITTED** (этот уровень изоляции, как показывает его название, гарантирует отсутствие феномена «грязного» чтения).



На этом рисунке показано, что в момент времени t_0 были образованы две транзакции T_1 и T_2 . В момент времени t_1 транзакция T_2 выполняет операцию чтения некоторого объекта базы данных O (например, производит выборку строки из таблицы с указанием значения первичного ключа). В момент времени t_2 ($t_2 > t_1$) транзакция T_1 изменяет объект O (модифицирует или даже удаляет). В момент времени t_3 ($t_3 > t_2$) транзакция T_2 повторно считывает объект O и обнаруживает, что он изменился или вовсе отсутствует.

Другими словами, в транзакции T_2 повторное выполнение выборки объекта базы данных O дало результат, отличный от результата первого выполнения (отсюда и происходит термин «неповторяемое чтение»).

ФЕНОМЕН ФАНТОМОВ

В SQL феномен фантомов может наблюдаться у транзакций, выполняемых на уровне изоляции *REPEATABLE READ* (этот уровень изоляции, как показывает его название, гарантирует отсутствие феномена неповторяемого чтения).



На этом рисунке показано, что в момент времени t_0 были образованы две транзакции, T1 и T2. В момент времени t_1 транзакция T2 выполняет операцию выборки строк из таблицы R по условию c. В момент времени t_2 ($t_2 > t_1$) транзакция T1 выполняет над таблицей R операцию обновления (вставки или модификации строк), в результате которой в таблице R появляются дополнительные строки, удовлетворяющие условию c. В момент времени t_3 ($t_3 > t_2$) транзакция T2 повторно выполняет операцию выборки строк из таблицы R по условию c и обнаруживает наличие в результате дополнительных фантомных строк.

УРОВНИ ИЗОЛЯЦИИ ТРАНЗАКЦИЙ И ФЕНОМЕНЫ



Наконец, для транзакций, выполняемых на уровне изоляции `SERIALIZABLE`, невозможно и проявление феномена фантомов. Термин *serializable* (серуализуемый) используется по той причине, что при работе на данном уровне изоляции суммарный эффект выполнения набора транзакций $\{T_1, T_2, \dots, T_n\}$ идентичен эффекту некоторого последовательного выполнения этих транзакций. Это означает предельную изолированность транзакций.

Уровень	«Грязное» чтение	Неповторяемое чтение	Фантомы
READ UNCOMMITTED	Возможно	Возможно	Возможны
READ COMMITTED	Невозможно	Возможно	Возможны
REPEATABLE READ	Невозможно	Невозможно	Возможны
SERIALIZABLE	Невозможно	Невозможно	Невозможны

ЗАВЕРШЕНИЕ ТРАНЗАКЦИЙ



Для завершения транзакции всегда требуется выполнение одного из двух операторов COMMIT (фиксация транзакции) или ROLLBACK (откат транзакции), которые имеют следующий синтаксис:

```
COMMIT [ WORK ] [ AND [ NO ] CHAIN ]  
ROLLBACK [ WORK ] [ AND [ NO ] CHAIN ]  
[ TO SAVEPOINT savepoint_name ]
```

При желании завершить транзакцию таким образом, чтобы все произведенные ею изменения были навсегда сохранены в базе данных, следует завершать транзакцию оператором COMMIT (как видно из синтаксиса, допускается эквивалентный вид COMMIT WORK). Если требуется завершить транзакцию с аннулированием всех произведенных изменений, то нужно использовать оператор ROLLBACK (ROLLBACK WORK).

При отсутствии явного указания в операторе завершения транзакции, подразумевается наличие раздела AND NO CHAIN, и новая транзакция не образуется. Если же раздел AND CHAIN присутствует, то немедленно после завершения выполнения COMMIT или ROLLBACK текущей транзакции образуется новая транзакция, наследующая все характеристики завершенной транзакции.



Допускается установка в одной транзакции нескольких последовательных точек сохранения. При установке каждой точки сохранения ей назначается некоторое (локальное в пределах транзакции) имя, которое в дальнейшем может использоваться в операции ROLLBACK для задания точки частичного отката транзакции. Если последовательно устанавливаются две точки сохранения SP1 и SP2 и затем выполняется операция ROLLBACK TO SAVEPOINT SP1, то восстановление производится до SP1 (через SP2), и точка сохранения SP2 «забывается».

Для установления точки сохранения используется оператор SAVEPOINT с очевидным синтаксисом

SAVEPOINT savepoint_name

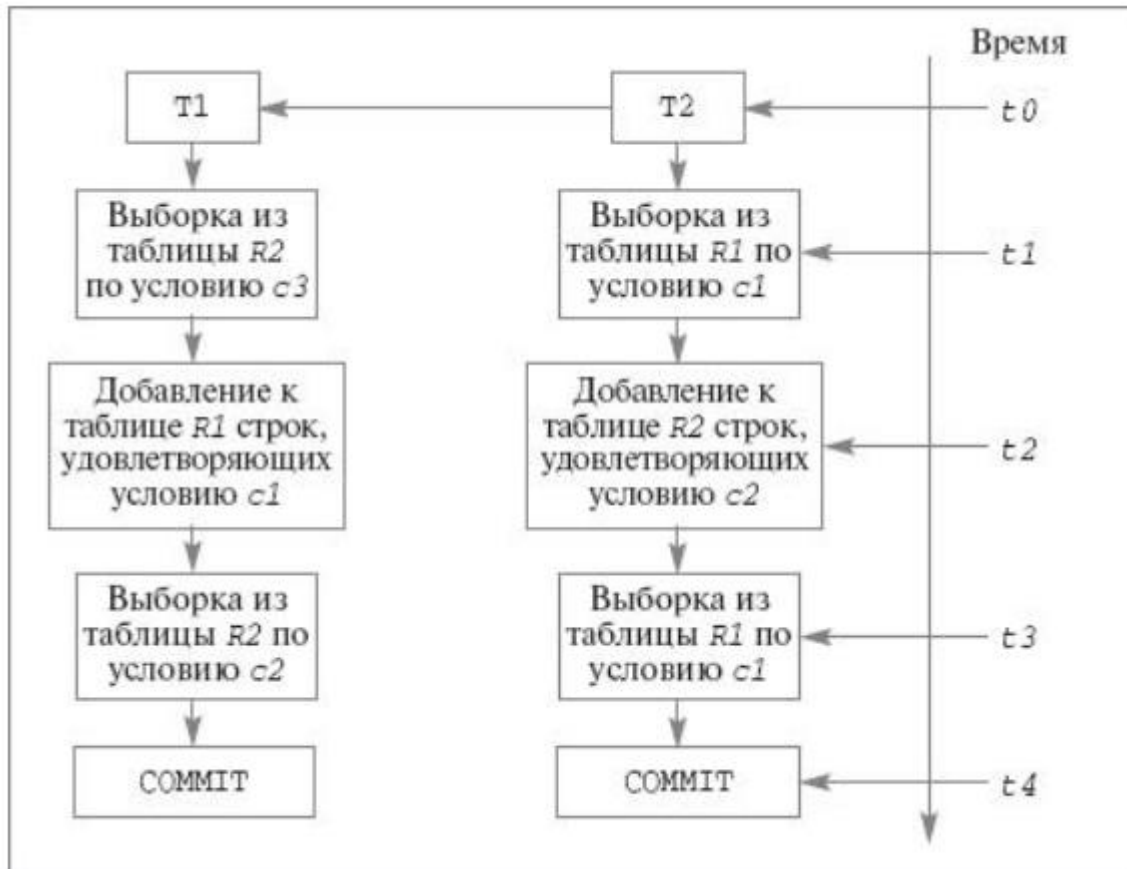
Можно также отказаться от ранее установленной точки сохранения, удалив ее из контекста транзакции. Для этого предназначен оператор RELEASE, синтаксис которого также очевиден:

RELEASE SAVEPOINT savepoint_name

ВЗАИМНЫЕ ФАНТОМЫ



Пусть имеются две транзакции T1 и T2, которые выполняются в режиме изоляции SERIALIZABLE



Транзакции работают в наивысшем режиме изолированности. Эффект их выполнения должен быть эквивалентен эффекту некоторого последовательного выполнения транзакций T1 и T2. Но попробуйте придумать какой-либо корректный способ одновременного выполнения этих транзакций, который привел бы к эффекту их последовательного выполнения. Другими словами, для грамотного использования механизма транзакций на уровне языка SQL необходимо знать, каким образом данный механизм реализован в используемой СУБД.

